



Project Overview — Technical & Logical Summary

What it does, how each part is built, and the reasoning behind every major decision.

1 · WHAT IT IS

A security-operations web app that connects to your endpoint tools (EDR / XDR / AV / SIEM), pulls their device inventories via API, **normalizes everything into one schema**, and presents a unified, historical, role-restricted view of **endpoint compliance** across every vendor and site.

- **Backend:** Laravel 13 + MySQL — REST API, Sanctum SPA auth.
- **Frontend:** Next.js 16 (App Router) + Tailwind v4 + shadcn-style UI + Recharts.
- **Deployment:** two subdomains — API and dashboard — behind nginx.

2 · THE PROBLEM & THE LOGIC

Problem. Orgs run several endpoint tools across multiple sites — each with its own console, login, permission model and data format. There's no single answer to *"are all endpoints protected, current, and checking in — everywhere, now?"* and no safe way to let L1/auditors/clients see posture without full console access.

The 5 logical pillars the whole system implements:

1. **Normalize** every vendor into one schema → comparison & totals become possible.
2. **Snapshot** every pull → history & trends, not just "now".
3. **Scope** by site / source / everything → one dataset, many views.
4. **Restrict by role** → monitoring (read-only) separated from action (the console).
5. **Encrypt the only real secret** (the API key) to a high standard → the tool earns the right to hold it.

3 · HIGH-LEVEL ARCHITECTURE

```
Browser (SPA)      Frontend — Next.js (Node server)
dashboard.<domain> → App Router pages, React Query, Recharts widgets, builder
                    | fetch (credentials: include) — Sanctum cookie + X-XSRF-TOKEN
                    v
api.<domain>         Backend — Laravel 13 (PHP-FPM)
                    Controllers → Services → Eloquent → MySQL
Scheduler (cron) → sources:refresh-due (every minute)
                    | HTTP per vendor API
                    v
CrowdStrike · Defender · SentinelOne · Trend Micro · Cortex XDR ·
Cisco AMP · Elastic · Wazuh · Sophos · Generic
```

Why cookie-based SPA (not bearer tokens): httpOnly session cookies + CSRF are safer than JS-stored tokens (XSS can't read httpOnly cookies). The two subdomains share cookies via `SESSION_DOMAIN=.<root-domain>`.

4 · TECH STACK (AND WHY)

LAYER	CHOICE	WHY
API	Laravel 13	Batteries-included auth/validation/queue/scheduler/ORM; secure fast
DB	MySQL	Relational fit for snapshots/endpoints/audit; ubiquitous
Auth	Sanctum (SPA cookies)	First-party session auth without JS-readable tokens
Hashing	Argon2id	OWASP-recommended, memory-hard
MFA	TOTP (google2fa)	Authenticator-app standard
Key crypto	AES-256-GCM (OpenSSL)	Authenticated encryption for connector secrets
UI	Next.js 16 App Router	Modern React, SSR server, file routing
Styling	Tailwind v4 + shadcn-style	Fast, consistent dark console aesthetic
Charts	Recharts	Declarative, composable
Data fetching	TanStack Query	Caching, invalidation, background refetch
Grid builder	react-grid-layout	Drag/resize widgets

5 · END-TO-END DATA FLOW

```
Connect a source (wizard) → ApiSource row (+ encrypted / session-only secret)
Scheduler or "Refresh" → IngestService.run(source, secret)
ConnectorFactory.make() → Generic / CrowdStrike / Wazuh connector
(auth resolve + paginate + data_path extract)
Raw vendor records → Normalizer (map fields, derive health/compliance)
Normalized endpoints → Summarizer (rollups) + DB write:
    • snapshots (captured_at, summary JSON)
    • endpoints (one row per device)
Frontend reads via InsightsController (scope-aware):
    summary / aggregate / trends / data / evaluate → widgets, tables, rule stats
```

Logic: ingestion and presentation are decoupled — ingestion produces a durable normalized record; presentation only ever reads it, so the UI is fast, vendor-agnostic, and historical by construction.

6 · FEATURE CATALOGUE — WHAT · HOW · WHY

6.1 Authentication & MFA

What — Email/password login + optional TOTP 2FA with recovery codes.

How — Two-step session: if MFA on, server holds a pending user in session and returns `mfa_required`, then verifies a TOTP/recovery code before `Auth::login()` + session regenerate. Argon2id hashing, auto-rehash on login.

Why — A stolen password alone can't log in; recovery codes prevent lockout if the authenticator is lost.

6.2 Role-based access control

What — Three roles — Admin / Analyst / Viewer.

How — `role` column + `EnsureRole` middleware + `canManage()/isAdmin()` helpers. Viewers read-only; analysts manage sources/dashboards; admins manage users.

Why — Least privilege — monitoring (Viewer) is separated from action, enabling safe access for L1/auditors/clients.

6.3 Login-IP security & audit

What — Capture login IP, baseline known IPs, red-flag a new IP, track active time, lock out brute force, full audit log.

How — `LoginSecurityService` records `login_events` + `known_ips` (first IP trusted; later IP sets `ip_flagged`), lockout after N fails; `TrackActivity` refreshes last-seen/IP; `AuditLogger` writes `audit_logs`.

Why — Takeover/insider risk surfaces as anomalies; the audit trail gives accountability & compliance evidence.

6.4 Connector engine (ingestion core)

What — Connect to any REST API; presets for 10 platforms (Generic, CrowdStrike, Defender, SentinelOne, Trend Micro, Cortex XDR, Cisco AMP, Elastic, Wazuh, Sophos).

How — `ConnectorFactory` picks Generic / CrowdStrike (two-step) / Wazuh (token-exchange). `AbstractConnector` resolves auth (Bearer / API-key / Basic / OAuth2-CC), extracts via `data_path`, paginates (none/offset/page/cursor). `VendorPresets` ship config + field mappings + an in-wizard "how to get the API key" guide.

Why — One configurable engine + presets = broad coverage without per-vendor code; the generic mapper future-proofs any tool.

6.5 Secret encryption (trust core)

What — API keys encrypted; two modes — Save (at rest) or Require every login (never stored).

How — `SecretBox` = AES-256-GCM (authenticated) with a dedicated `DATA_ENCRYPTION_KEY` (independent rotation). Saved = ciphertext in DB; per-login = encrypted session only (`SessionSecretVault`), wiped on logout.

Why — The API key is the only true secret; bank-grade protection + a never-stored option removes the "is the dashboard a risk?" objection. (Trade-off: per-login disables background refresh.)

6.6 Normalization

What — Turn each vendor record into one shape (hostname, OS, agent version, last seen, health, compliance, ip/mac, raw).

How — `Normalizer` reads dot-path field mappings, normalizes OS names, parses timestamps, then derives connectivity from last-seen recency ($\leq 24h$ online, $\leq 7d$ stale, else offline) and compliance from agent-present + recency; keeps original `raw`.

Why — Consistent semantics across vendors is what makes one compliance % and cross-tool charts meaningful.

6.7 Snapshots & trends

What — Every refresh stored as a point-in-time snapshot with a pre-aggregated summary; trends across snapshots.

How — `IngestService` writes a `snapshots` row (summary JSON) + `endpoints` rows; `Summarizer` builds rollups at write time; old per-endpoint detail pruned past a retention window.

Why — Pre-aggregating at write makes trend/dashboard reads cheap (no scanning millions of rows) — fast charts, provable history.

6.8 Sites & scope aggregation

What — Group sources into Sites; view All sites, one site, or one source.

How — `sites` + `api_sources.site_id`; `InsightsController` resolves a `scope` (`all|site:{id}|source:{id}`), merges latest-snapshot summaries, unions endpoints, and date-buckets trends. Frontend `useScope` + one-click site chips.

Why — Multi-location/tenant monitoring in one pane; one ingestion effort → per-site and estate-wide views.

6.9 Default dashboard

What — Out-of-the-box widgets (totals, online/offline, compliance gauge, OS pie, health donut, compliance bar, agent-version bar, trend line, table).

How — `DefaultDashboard::layout()` defines a 12-col grid; auto-created per user on first load.

Why — Instant value with zero config; a sensible baseline users then tweak.

6.10 Custom dashboard builder

What — Drag/resize/add/remove widgets; choose type/metric/field/series; saved per user.

How — `/builder` via react-grid-layout; layout persisted as JSON in `dashboards.layout`; loads every login.

Why — Different roles care about different things; saved layouts mean nobody reconfigures each session.

6.11 Custom rule engine

What — Stat/gauge widgets that count endpoints matching your own logic (e.g. last seen > 2 days, Windows AND non-compliant).

How — RuleBuilder produces `{match, conditions:[{field,op,value}]}`; `POST /insights/evaluate` runs `RuleEvaluator`, translating conditions into a scoped query (time cutoffs, eq/neq/contains/in/empty, AND/OR).

Why — Vendor dashboards have fixed buckets; real policy must be user-expressible, across all tools, on demand.

6.12 Endpoint data table

What — Raw normalized records with search, filters, sort, pagination, CSV export.

How — `EndpointTable` calls scope-aware `/insights/data`; exports current view client-side.

Why — Charts answer "how many"; the table answers "which ones" — needed to act.

6.13 Admin CMS

What — List/create users, change role/status, reset passwords (temp + force change), clear IP flags, unlock, reset MFA; view login activity + audit log.

How — `AdminController` under `role:admin`; `/admin` tabs + manage-user dialog (known IPs, recent sign-ins).

Why — Central, accountable administration without DB access; supports least-privilege & incident response.

6.14 System health

What — Live status of DB, cache, queue, scheduler heartbeat, sources (failing/stale), security posture.

How — `HealthController` probes each subsystem; the scheduler writes a cache heartbeat so a dead cron is detectable.

Why — A monitoring tool must monitor itself — silent ingestion failure is worse than a visible error.

6.15 Scheduling

What — Auto-refresh each source on its own interval (1h/2h/...).

How — `sources:refresh-due` runs every minute (cron → `schedule:run`), refreshes due saved-secret sources inline, writes the heartbeat.

Why — Fresh data without manual pulls; inline (no queue) keeps deploy simple, with a documented path to queues for scale.

7 · DATA MODEL (MYSQL)

TABLE	PURPOSE
users	accounts + role, MFA secret/recovery, IP/lockout fields
sites	named groupings of sources
api_sources	connector config: vendor, base_url, auth, request/mappings, interval, secret mode + ciphertext
source_runs	each refresh attempt (status, duration, records, error)
snapshots	point-in-time pull with pre-aggregated summary JSON
endpoints	one normalized device row per snapshot (+ raw payload)
dashboards	per-user saved widget layouts
login_events	every login attempt (success/fail, new-IP, reason)
known_ips	per-user trusted IP baseline
audit_logs	who did what, when, from where

8 · API SURFACE

- **Auth:** POST /login, /login/mfa, /logout, GET /me, PUT /password
- **MFA:** POST /mfa/setup|confirm|disable|recovery-codes
- **Sources:** GET /sources/presets, POST /sources/test, CRUD /sources, POST /sources/{id}/refresh|unlock, GET /sources/{id}/runs
- **Sites:** CRUD /sites, POST /sites/{id}/assign
- **Insights (scope-aware):** GET /insights/summary|aggregate|trends|data, POST /insights/evaluate
- **Dashboards:** GET /dashboards, /dashboards/default, CRUD /dashboards
- **Admin (role:admin):** /admin/users..., /admin/login-events, /admin/audit-logs
- **Health:** GET /health

All non-auth routes sit behind auth:sanctum + activity tracking.

9 · SECURITY MODEL (TECHNICAL + LOGICAL)

CONTROL	IMPLEMENTATION	LOGIC
Password storage	Argon2id, tunable, auto-rehash	Memory-hard hashing resists cracking
MFA	TOTP + hashed recovery codes	Stolen password ≠ access
Session	Sanctum cookies, encrypted, CSRF	No JS-readable tokens; CSRF blocked
API-key secrecy	AES-256-GCM, dedicated key, never-stored option	Protect the only real secret; honest trade-offs
RBAC	role middleware + helpers	Least privilege; monitoring ≠ action
IP anomaly	known-IP baseline + red flag	Surface takeover/insider risk
Brute force	attempt counter + lockout	Slow credential stuffing
Audit	append-only audit_logs	Accountability & compliance evidence
Transport	HTTPS + secure cookies	Confidentiality/integrity in transit

The "trust case": a monitoring tool built to the standards it monitors.

10 · DEPLOYMENT TOPOLOGY

```

dashboard.<domain> (site)
  nginx 443 —proxy→ Node :PORT
  (PM2 `next start`)

api.<domain> (site)
  nginx 443 → nginx :8080 → PHP-FPM
  root = Laravel /public
  cron: * * * * * artisan schedule:run
  
```

- **Frontend:** npm run build, run via PM2 on a unique port, nginx reverse-proxies /. NEXT_PUBLIC_API_URL baked at build.
- **Backend:** doc-root = Laravel public/; PHP-FPM; scheduler cron every minute.
- **Cross-subdomain auth:** SESSION_DOMAIN=.<root-domain>, SameSite=Lax, secure cookies, CORS allows the frontend origin with credentials.

11 · KEY DESIGN DECISIONS & TRADE-OFFS

1. **Cookie SPA over bearer tokens** — safer vs XSS; cost is cross-subdomain cookie config.
2. **Pre-aggregated snapshots** — fast trends; cost is storage + a retention policy.
3. **Generic connector + presets** over per-vendor SDKs — broad coverage, less code; cost is occasional field-mapping tweaks.
4. **Inline scheduled ingestion** (no queue) — simplest reliable deploy; documented path to queued workers for scale.
5. **Report/visualize only, not respond** — complements vendor consoles rather than duplicating detection/response; contains scope & risk.

12 · LIMITATIONS & FUTURE WORK

- Reports & visualizes coverage/compliance — does **not** perform detection/response (that stays in the consoles).
- Data freshness = refresh interval; accuracy = each vendor API's accuracy.
- Per-login secret mode can't auto-refresh in the background (by design).
- Next steps: queued ingestion for huge fleets, per-site stat cards on All-sites, alerting on rule breaches, SSO/SAML, a saved reusable-rule library.